1/23/2026

FINANCIAL FRAUD DETECTION

Anomaly Detection in Credit Card Transactions

KAPIL KUMAR MANDAWARIA

Financial Fraud Detection: Anomaly Detection in Credit Card Transactions

Project Overview

In the modern financial ecosystem, the transition to digital payments has accelerated the volume of transactions, but it has also increased the surface area for fraudulent activity. **SecureGuard Financial Solutions** addresses this by moving beyond traditional "reactive" security to a "proactive" analytical model. This project centers on building a robust detection system that identifies irregularities in credit card usage before they result in significant financial liability.

Project Statement and Scope

Design a fraud detection system to identify fraudulent credit card transactions in real-time. Fraud manifests in diverse ways—unauthorized card usage, identity theft, and atypical spending spikes. Standard threshold-based alerts often lead to high "false positive" rates; therefore, this project utilizes **Statistical Anomaly Detection** and **Geospatial Mapping** to refine the accuracy of fraud flagging.

The project encompasses the end-to-end development of an analytical pipeline, including:

- **Data Integration:** Processing historical transaction data containing timestamps, amounts, and geographical locations.
- **Statistical Outlier Detection:** Utilizing **Box-and-Whisker plots** to isolate transaction amounts that fall outside the interquartile range (IQR), effectively identifying extreme spending values.
- **Geospatial Analysis:** Visualizing the geographical distribution of fraudulent versus legitimate transactions to identify high-risk locations.
- **Trend Analysis:** Monitoring monthly and weekly transaction volumes and amounts to establish a "baseline" of normal behavior.

Core Project Objectives

- **Minimize Financial Loss:** Detect and halt fraudulent activities in real-time.
- **Enhance Decision Making:** Provide security analysts with a centralized Tableau Dashboard that integrates spatial, temporal, and statistical views of fraud.
- **Maintain Consumer Trust:** Reduce the friction of false positives while ensuring high-risk transactions are blocked instantly.

Tools Used

- **Excel** – for statistical summaries and basic visualizations
- **SQL** – for data preprocessing and aggregations
- **Python (Pandas, NumPy, Matplotlib, Seaborn)** – for exploratory data analysis (EDA)
- **Tableau** – for interactive dashboards

1. *Excel Tasks:*

a. Create a statistical summary on amt and city_pop

amt	
Mean	70.44214816
Standard Error	0.2600671
Median	47.57
Mode	1.04
Standard Deviation	162.2039152
Sample Variance	26310.11011
Kurtosis	3868.242869
Skewness	40.27383233
Range	27389.12
Minimum	1
Maximum	27390.12
Sum	27402136.52
Count	389002

city_pop	
Mean	88680.84286
Standard Error	482.9404869
Median	2456
Mode	606
Standard Deviation	301210.1017
Sample Variance	90727525376
Kurtosis	37.54431154
Skewness	5.587892446
Range	2906677
Minimum	23
Maximum	2906700
Sum	34497025233
Count	389002

1.Amount (amt) Interpretation:

- This dataset shows a highly right-skewed distribution, meaning most transactions are small, but there are some extremely large values pulling the averages up.
- Averages vs. Reality: The Mean (70.44) is much higher than the Median (47.57). This indicates that high-value "outliers" are inflating the average. In fact, the most common value (Mode) is only 1.04, suggesting a high frequency of very small transactions.
- Extreme Volatility: The Standard Deviation (162.20) is more than double the mean. This tells us the data is spread out significantly; transactions vary wildly from the average.

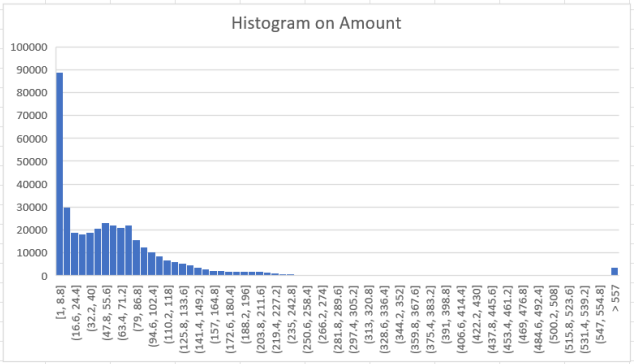
2.City Population (city_pop) Interpretation

- The population data also shows significant inequality, but it is distributed differently than the transaction amounts.
- Massive Skew: The Mean population is 88,680, but the Median is only 2,456. This is a classic sign of a dataset where a few massive metropolises (up to 2.9 million people) coexist with hundreds of thousands of very small towns or villages.
- The "Typical" City: Since the Median is 2,456 and the Mode is 606, we can conclude that the vast majority of your data points represent very small towns, even though the "average" suggests a mid-sized city.

b. Plot a histogram on amt

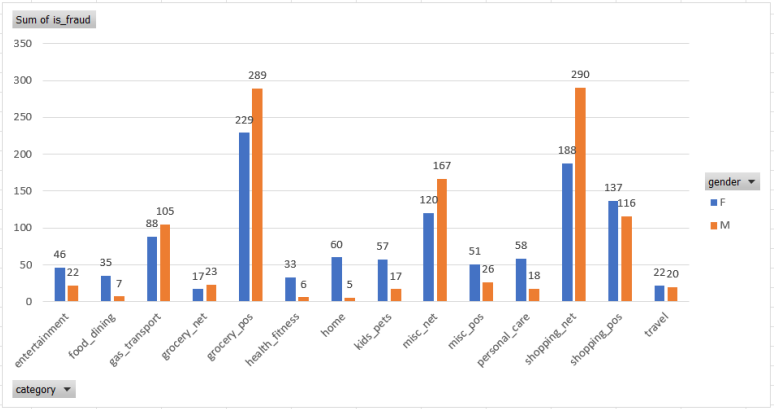
Bin	Frequency	Cumulative %
1	70	0.02%
27.77333	145086	37.31%
54.54667	70040	55.32%
81.32	73060	74.10%
108.0933	39617	84.29%
134.8667	21061	89.70%
161.64	11779	92.73%
188.4133	6717	94.45%
215.1867	6060	96.01%
241.96	3327	96.87%
268.7333	1682	97.30%
295.5067	1208	97.61%
322.28	1032	97.88%
349.0533	759	98.07%
375.8267	568	98.22%
402.6	469	98.34%
429.3733	416	98.44%
456.1467	492	98.57%
482.92	549	98.71%
509.6933	528	98.85%
536.4667	488	98.97%
563.24	348	99.06%
590.0133	267	99.13%
616.7867	221	99.19%

bin	Frequency
10	100712
20	26260
30	23370
40	24213
50	27035
100	117012
200	51652
500	14086
1000	3471
5000	1147
more than	44



c. Create a report showing number of Frauds by gender and category

Sum of is_fraud	Column Labels		
Row Labels	F	M	Grand Total
entertainment	46	22	68
food_dining	35	7	42
gas_transport	88	105	193
grocery_net	17	23	40
grocery_pos	229	289	518
health_fitness	33	6	39
home	60	5	65
kids_pets	57	17	74
misc_net	120	167	287
misc_pos	51	26	77
personal_care	58	18	76
shopping_net	188	290	478
shopping_pos	137	116	253
travel	22	20	42
Grand Total	**1141**	**1111**	**2252**



d. Show the top 3 states by the highest number of transactions

top 5 states by the highest number of transactions

Row Labels	Count of index
TX	28482
NY	25034
PA	23911
CA	17007
OH	13933
Grand Total	**108367**

e. Is there a correlation between transaction amount and city population?

correlation between transaction amount and city population	**0.007270752**	=CORREL(cc_data!F2:F389003,cc_data!P2:P389003)

Interpretation: The result will be between **-1 and 1**.

Close to 1: As city population increases, transaction amounts tend to increase.

Close to 0: There is no linear relationship between city size and transaction amount.

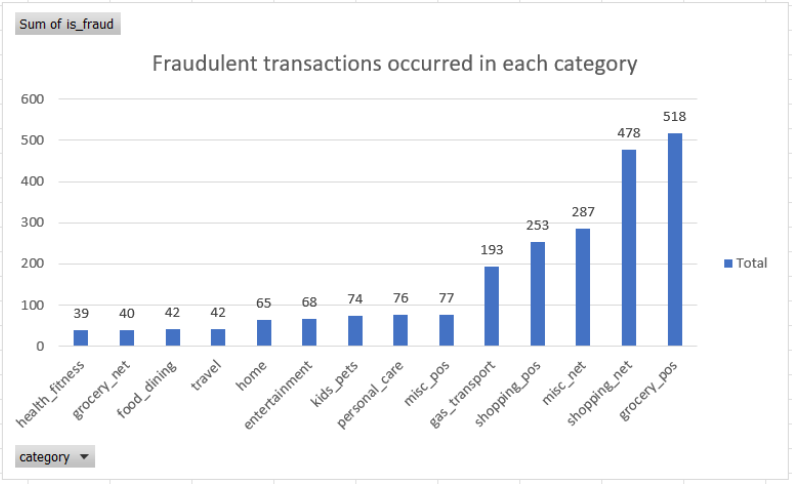
Close to -1: As city population increases, transaction amounts tend to decrease.

	amt	*city_pop*
amt	1	
city_pop	0.007270752	1

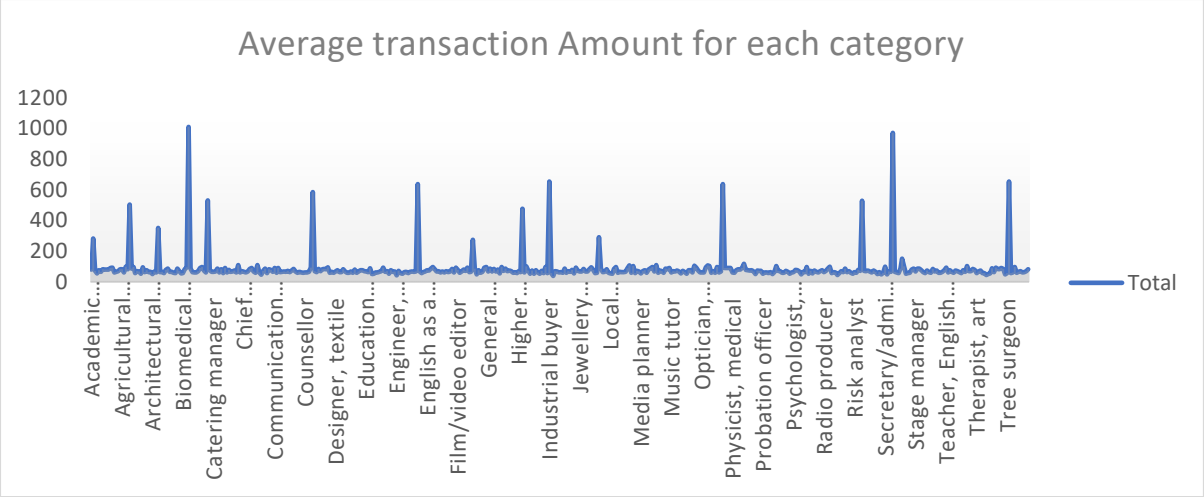
f. How many fraudulent transactions occurred in each category?

Fraudulent Transactions occurred in each Category

Row Labels	Sum of is_fraud
health_fitness	39
grocery_net	40
food_dining	42
travel	42
home	65
entertainment	68
kids_pets	74
personal_care	76
misc_pos	77
gas_transport	193
shopping_pos	253
misc_net	287
shopping_net	478
grocery_pos	518
Grand Total	**2252**



g. How does the average transaction amount vary between different job roles?



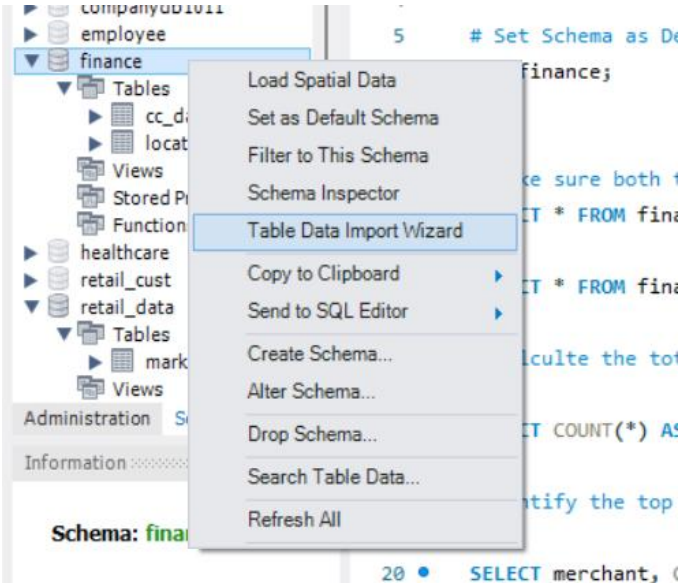
2. *SQL Tasks:*

Data Loading

- Create a schema named **finance**, set **finance** as the default schema, and create tables with cc_data.csv and location_data.csv

```
1    # Create Scema named Finance
2
3 •  CREATE SCHEMA finance;
4
5    # Set Schema as Default
6 •  USE finance;
7
```

- Imported table **cc_data.csv** and **location_data.csv** using "Table Data Import Wizard"



Data Exploration with SQL:

- Calculate the total number of transactions in the cc_data table

```
14    # Calculte the total number of transaction in the cc_data table.
15
16    SELECT COUNT(*) AS total_transaction FROM cc_data;
17
18    #Identify the top 10 most frequent merchants in the cc_data table.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_transaction
47540

- Identify the top 10 most frequent merchants in the cc_data table

```
18    #Identify the top 10 most frequent merchants in the cc_data table.
19
20    SELECT merchant, COUNT(*) AS transaction_count
21    FROM cc_data
22    GROUP BY merchant
23    ORDER BY transaction_count DESC
24    LIMIT 10;
25
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

merchant	transaction_count
fraud_Kilback LLC	155
fraud_Cormier LLC	148
fraud_Kuhn LLC	121
fraud_Boyer PLC	121
fraud_Schumm PLC	119
fraud_Dickinson Ltd	118
fraud_Erdman-Kertzmann	118
fraud_Sporer Inc	116

- Find the average transaction amount for each category of transactions in the cc_data table

```
27    # Find the average transaction amount for each category of transactions in the cc_data table
28
29    select category, AVG(amt) as average_transaction_amount
30    from cc_data
31    group by category
32    order by category;
33
34
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_transaction_amount
entertainment	64.97637526018434
food_dining	51.653456193353456
gas_transport	63.49238222040133
grocery_net	54.12217821782174
grocery_pos	117.56023740201583
health_fitness	54.01426646248085
home	59.011085932241464
kids_pets	57.8468948004836

- Determine the number of fraudulent transactions and the percentage of total transactions that they represent

```
35    # Determine the number of fraudulent transactions and the percentage of total transactions that they represent
36
37    SELECT
38        COUNT(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraudulent_count,
39        COUNT(*) AS total_transactions,
40        ROUND(
41            (COUNT(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) * 100.0) / COUNT(*), 4) AS fraud_percentage
42    FROM cc_data;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

fraudulent_count	total_transactions	fraud_percentage
47540	47540	100.0000

- Join the cc_data and location_data tables to identify the latitude and longitude of each transaction

```
44    # Join the cc_data and location_data tables to identify the latitude and longitude of each transaction
45
46    SELECT cc.*, ld.lat, ld.long
47    FROM cc_data cc
48    JOIN location_data ld ON cc.cc_num = ld.cc_num;
49
50    # Identify the city with the highest population in the location_data table
51
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

index	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	city
839413	13-12-2019 12:36	60416207185	fraud_Schaefer Ltd	kids_pets	281.39	Mary	Diaz	F	9886 Anita Drive	Fort Washakie
126448	11-03-2019 04:16	60422928733	fraud_Friesen Inc	shopping_pos	2.53	Jeffrey	Powers	M	38352 Parrish Road Apt. 652	North Augusta
70285	11-02-2019 09:40	60416207185	fraud_Cartwright-Harris	grocery_pos	135.08	Mary	Diaz	F	9886 Anita Drive	Fort Washakie
1142925	21-04-2020 03:36	60422928733	fraud_Cummerata-Jones	gas_transport	72.8	Jeffrey	Powers	M	38352 Parrish Road Apt. 652	North Augusta
145698	19-03-2019 17:47	60427851591	fraud_Rau-Grant	kids_pets	49.41	Bradley	Martinez	M	3426 David Divide Suite 717	Burns Flat
605686	14-09-2019 18:51	60422928733	fraud_Reichel, Bradtke and Blanda	travel	8.11	Jeffrey	Powers	M	38352 Parrish Road Apt. 652	North Augusta
643152	30-09-2019 14:32	60495593109	fraud_Hirthe-Beier	health_fitness	33.72	Randall	Dillon	M	4440 George Mills Suite 591	Dallas

- Identify the city with the highest population in the location_data table

```
50    # Identify the city with the highest population in the location_data table
51
52    SELECT
53        city,
54        MAX(city_pop) AS highest_population
55    FROM cc_data
56    GROUP BY city
57    ORDER BY highest_population DESC limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

city	highest_population
Houston	2906700

- Find the earliest and latest transaction dates in the cc_data table

```
60    # Find the earliest and latest transaction dates in the cc_data table
61
62    SELECT
63        MIN(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS earliest_transaction,
64        MAX(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS latest_transaction
65    FROM cc_data;
66
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

earliest_transaction	latest_transaction
2019-01-01 00:22:00	2020-06-21 11:37:00

Using Data Aggregation with SQL

- What is the total amount spent across all transactions in the cc_data table?

```
68    # What is the total amount spent across all transactions in the cc_data table?
69
70    SELECT SUM(amt) AS total_amount_spent FROM cc_data;
71
72    # How many transactions occurred in each category in the cc_data table?
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_amount_spent
3403719.670000004

- How many transactions occurred in each category in the cc_data table?

```
72    # How many transactions occurred in each category in the cc_data table?
73
74    SELECT category, COUNT(*) AS transaction_per_category
75    FROM cc_data
76    GROUP BY category
77    ORDER BY category;
78
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	transaction_per_category
entertainment	3363
food_dining	3310
gas_transport	4882
grocery_net	1717
grocery_pos	4465
health_fitness	3265
home	4457
kids_pets	4135

- What is the average transaction amount for each gender in the cc_data table?

```
79    # What is the average transaction amount for each gender in the cc_data table?
80
81    SELECT gender,  AVG(amt) AS average_transaction_amount
82    FROM cc_data
83    GROUP BY gender
84    order by gender;
85
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

gender	average_transaction_amount
F	71.52129430246447
M	71.68907280444061

➢ Which day of the week has the highest average transaction amount in the cc_data table?

```
86
87  # Which day of the week has the highest average transaction amount in the cc_data table?
88
89  SELECT
90      DAYNAME(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS day_of_week,
91      AVG(amt) AS average_transaction_amount
92  FROM cc_data
93  GROUP BY day_of_week
94  ORDER BY average_transaction_amount DESC
95  LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

day_of_week	average_transaction_amount
Monday	74.80758212170161
Tuesday	74.04196972274185
Thursday	72.90588575695345
Friday	71.94094754038822
Sunday	69.6260441240719

3. *Python Tasks (including EDA):*

Exploratory Data Analysis (EDA) with Python:

➢ Load the dataset and import required libraries.

```
[2]: import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     import scipy.stats as sc
```

```
[3]: # Load the dataset
     # Replace "your_dataset.csv" with the path to your dataset
     data = pd.read_csv(r"C:\Users\kalya\OneDrive\Documents\KAPIL_DATAanalysis\Capstone Project\Financial_Fraud_Detection_Datasets\cc_data.csv")
     data
```

[3]:

	index	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	...	lat	long	ci
0	151888	23-03-2019 03:06	3586008444788260	fraud_Ferry, Lynch and Kautzer	misc_net	1.24	Crystal	Fuller	F	000 Jennifer Mills	...	47.4974	-122.0107	
1	1185025	10-05-2020 11:28	4671727014157740	fraud_Bogisich Inc	grocery_pos	112.57	Kenneth	Edwards	M	3653 Ryan Crossroad	...	40.8618	-85.6067	
2	10818	07-01-2019 13:31	377993105397617	fraud_Mohr-Bayer	shopping_net	6.69	Nathan	Martinez	M	586 Thomas Cliffs	...	44.8755	-88.1555	
3	975275	30-01-2020 18:49	4710826438164840000	fraud_Langworth LLC	personal_care	100.69	Juan	Henry	M	9795 Lori Island Suite 346	...	48.8328	-108.3961	

Task1: What are the dimensions (number of rows and columns) of the dataset?

```
]: # Task1: What are the dimensions (number of rows and columns) of the dataset?
   # Dimensions of the dataset

   print("Dimensions (number of rows and columns) of the dataset:", data.shape)
```

```
Dimensions (number of rows and columns) of the dataset: (389002, 22)
```

Task 2: How many unique values are there in each categorical variable?

[6]:
```
# Task 2: How many unique values are there in each categorical variable?
# Unique values in each categorial variable

unique_values = data.select_dtypes(include=['object']).nunique()
print("Unique values in each categorical variable:")
print(unique_values)
```

```
Unique values in each categorical variable:
trans_date_trans_time    293627
merchant                    693
category                     14
first                       352
last                        481
gender                        2
street                      979
city                        890
state                        51
job                         492
dob                         964
trans_num                389002
dtype: int64
```

Task 3: What is the distribution of numerical variables in the dataset?

[7]:
```
# Task 3: What is the distribution of numerical variables in the dataset?
# Distribution of numerical variables

numerical_summary = data.describe()
print("Distribution of numerical variables:")
numerical_summary
```

```
Distribution of numerical variables:
```

[7]:

	cc_num	amt	zip	lat	long	city_pop	unix_time	merch_lat	merch_long	is_fraud
count	3.890020e+05	389002.000000	389002.000000	389002.000000	389002.000000	3.890020e+05	3.890020e+05	389002.000000	389002.000000	389002.000000
mean	4.191512e+17	70.442148	48818.064295	38.533121	-90.237664	8.868084e+04	1.349251e+09	38.531683	-90.236674	0.005789
std	1.311579e+18	162.203915	26879.383224	5.074596	13.745855	3.012101e+05	1.285085e+07	5.109400	13.757311	0.075866
min	6.041621e+10	1.000000	1257.000000	20.027100	-165.672300	2.300000e+01	1.325376e+09	19.029798	-166.669638	0.000000
25%	1.800429e+14	9.660000	26237.000000	34.620500	-96.798000	7.430000e+02	1.338751e+09	34.719394	-96.905445	0.000000
50%	3.521417e+15	47.570000	48174.000000	39.354300	-87.476900	2.456000e+03	1.349267e+09	39.361065	-87.446843	0.000000
75%	4.642255e+15	83.077500	72011.000000	41.940400	-80.158000	2.032800e+04	1.359460e+09	41.956012	-80.253831	0.000000
max	4.992346e+18	27390.120000	99783.000000	66.693300	-67.950300	2.906700e+06	1.371817e+09	67.064277	-66.956540	1.000000

Task 4: Are there any missing values in the dataset?

```python
# Task 4: Are there any missing values in the dataset?
# Missing values in the dataset

missing_values = data.isnull().sum()
print("Missing values in the dataset: ")
print(missing_values)
```

```
Missing values in the dataset:
trans_date_trans_time    0
cc_num                   0
merchant                 0
category                 0
amt                      0
first                    0
last                     0
gender                   0
street                   0
city                     0
state                    0
zip                      0
lat                      0
long                     0
city_pop                 0
job                      0
dob                      0
trans_num                0
unix_time                0
merch_lat                0
merch_long               0
is_fraud                 0
dtype: int64
```

Task 5: What are the summary statistics (mean, median, min, max, etc.) for numerical variables?

[9]:
```python
# Task 5:What are the summary statistics (mean, median, min, max, etc.) for numerical variables?
# Summary statistic for numerical variables

summary_statistics = data.describe(include='all')
print("Summary statistic for numerical variables: ")
summary_statistics
```

```
Summary statistic for numerical variables:
```

[9]:

	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street	city	...	lat	lc
count	389002	3.890020e+05	389002	389002	389002.000000	389002	389002	389002	389002	389002	...	389002.000000	389002.000(
unique	293627	NaN	693	14	NaN	352	481	2	979	890	...	NaN	N
top	22-12-2019 18:12	NaN	fraud_Kilback LLC	gas_transport	NaN	Christopher	Smith	F	1652 James Mews	Birmingham	...	NaN	N
freq	8	NaN	1305	39633	NaN	7982	8511	213231	980	1740	...	NaN	N
mean	NaN	4.191512e+17	NaN	NaN	70.442148	NaN	NaN	NaN	NaN	NaN	...	38.533121	-90.237(
std	NaN	1.311579e+18	NaN	NaN	162.203915	NaN	NaN	NaN	NaN	NaN	...	5.074596	13.745(
min	NaN	6.041621e+10	NaN	NaN	1.000000	NaN	NaN	NaN	NaN	NaN	...	20.027100	-165.672:
25%	NaN	1.800429e+14	NaN	NaN	9.660000	NaN	NaN	NaN	NaN	NaN	...	34.620500	-96.798(
50%	NaN	3.521417e+15	NaN	NaN	47.570000	NaN	NaN	NaN	NaN	NaN	...	39.354300	-87.476(
75%	NaN	4.642255e+15	NaN	NaN	83.077500	NaN	NaN	NaN	NaN	NaN	...	41.940400	-80.158(
max	NaN	4.992346e+18	NaN	NaN	27390.120000	NaN	NaN	NaN	NaN	NaN	...	66.693300	-67.950:

Task 6: Is there any correlation between numerical variables? If so, how strong is the correlation?

[10]:
```
# Task 6: Is there any correlation between numerical variables? If so, how strong is the correlation?
# Correlation between numerical variables

correlation_matrix = data.select_dtypes([float,int]).corr()
print("Correlation between numerical varables: ")
correlation_matrix
```

```
Correlation between numerical varables:
```

[10]:

	cc_num	amt	zip	lat	long	city_pop	unix_time	merch_lat	merch_long	is_fraud
cc_num	1.000000	0.002975	0.040486	-0.059182	-0.047587	-0.009125	0.001333	-0.058560	-0.047549	-0.001280
amt	0.002975	1.000000	0.001756	-0.000445	-0.000517	0.007271	0.000780	-0.000314	-0.000578	0.210706
zip	0.040486	0.001756	1.000000	-0.112567	-0.910295	0.078373	0.000905	-0.111689	-0.909470	-0.001220
lat	-0.059182	-0.000445	-0.112567	1.000000	-0.016080	-0.155972	-0.000150	0.993599	-0.015994	0.002643
long	-0.047587	-0.000517	-0.910295	-0.016080	1.000000	-0.053461	-0.001608	-0.016010	0.999119	0.001376
city_pop	-0.009125	0.007271	0.078373	-0.155972	-0.053461	1.000000	-0.001766	-0.154767	-0.053468	0.001176
unix_time	0.001333	0.000780	0.000905	-0.000150	-0.001608	-0.001766	1.000000	-0.000354	-0.001582	-0.006562
merch_lat	-0.058560	-0.000314	-0.111689	0.993599	-0.016010	-0.154767	-0.000354	1.000000	-0.015932	0.002406
merch_long	-0.047549	-0.000578	-0.909470	-0.015994	0.999119	-0.053468	-0.001582	-0.015932	1.000000	0.001351
is_fraud	-0.001280	0.210706	-0.001220	0.002643	0.001376	0.001176	-0.006562	0.002406	0.001351	1.000000

Most correlations are weak: Many values are close to 0, indicating little to no linear relationship.

Moderate Positive Correlation

- amt **vs** is_fraud **= 0.211** Indicates that higher transaction amounts are somewhat more likely to be fraudulent. This is a meaningful insight for fraud detection models.

Low or No Correlation

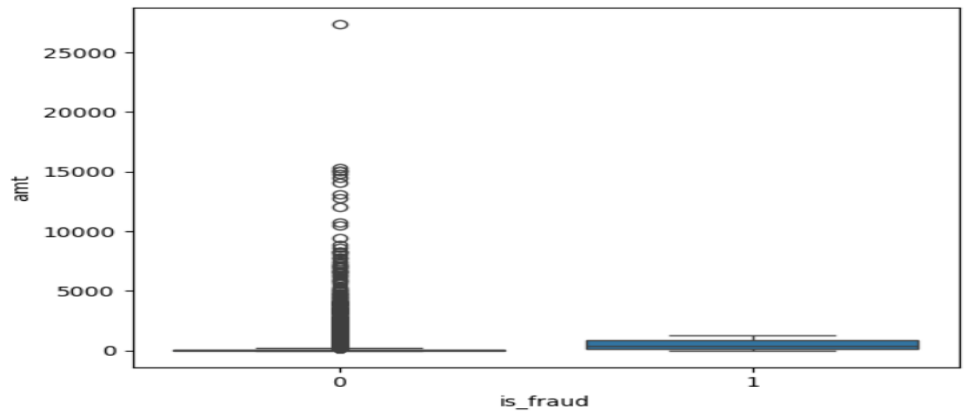
- Variables like cc_num, unix_time, and city_pop show very weak correlations with is_fraud, suggesting they may not be strong predictors on their own.

Strong Positive Correlations

- lat ↔ merch_lat (≈0.99)
- long ↔ merch_long (≈0.999) These show that merchant location is almost identical to the cardholder's location, which makes sense geographically.

Task 7: How does the distribution of an amt differ across is_fraud categories?

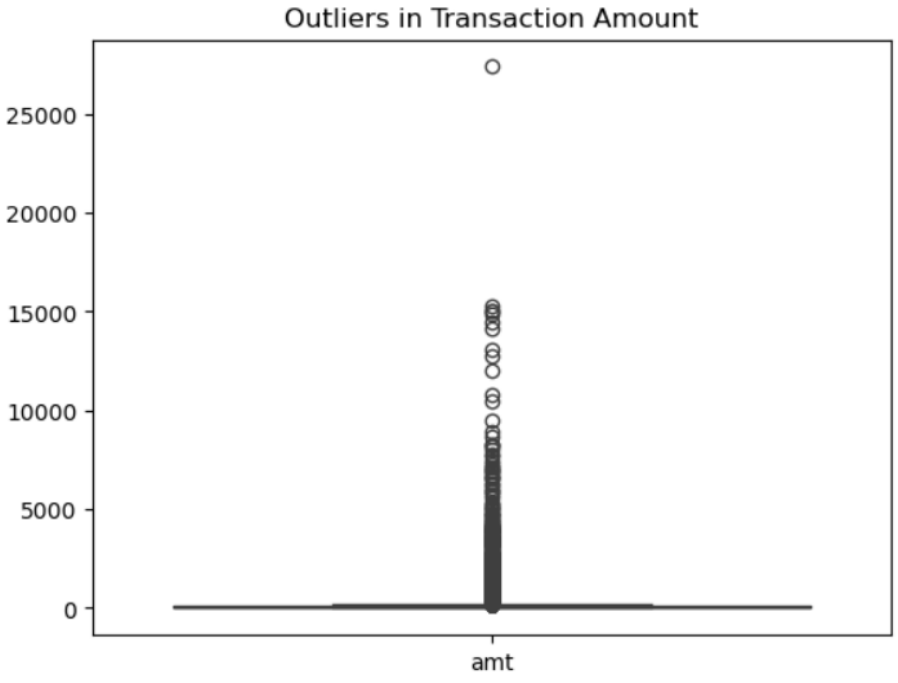
```
# Task 7: How does the distribution of an amt differ across is_fraud categories?

sns.boxplot(x='is_fraud', y='amt', data=data)
plt.show()
```



Task 8: Are there any outliers in the city_pop and amt?

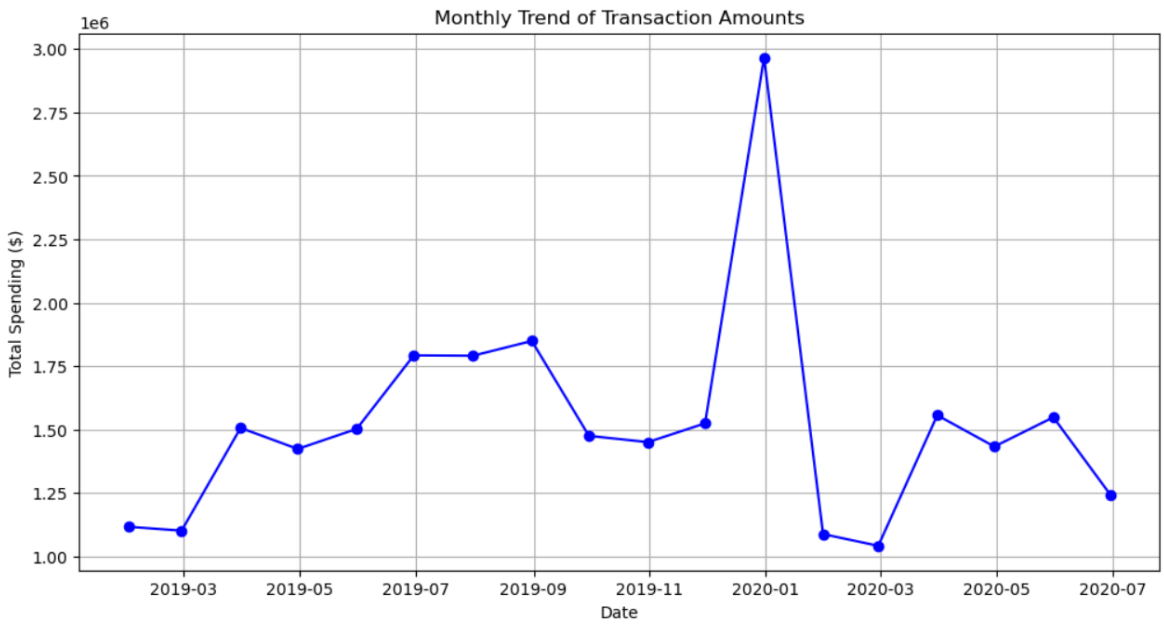
```
def find_outliers_iqr(data, column):
    # Calculate Q1 (25th percentile) and Q3 (75th percentile)
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1

    # Define bounds for outliers
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]

    return outliers, lower_bound, upper_bound

# 1. Detect outliers for 'amt'
amt_outliers, amt_lower, amt_upper = find_outliers_iqr(data, 'amt')
print(f"--- 'amt' Outlier Analysis ---")
print(f"Anything above {amt_upper:.2f} is an outlier.")
print(f"Number of 'amt' outliers: {len(amt_outliers)}")
print(f"Percentage of dataset: {(len(amt_outliers)/len(data))*100:.2f}%\n")

# 2. Detect outliers for 'city_pop'
pop_outliers, pop_lower, pop_upper = find_outliers_iqr(data, 'city_pop')
print(f"--- 'city_pop' Outlier Analysis ---")
print(f"Anything above {pop_upper:.2f} is an outlier.")
print(f"Number of 'city_pop' outliers: {len(pop_outliers)}")
print(f"Percentage of dataset: {(len(pop_outliers)/len(data))*100:.2f}%")


# The dots beyond the "whiskers" are your outliers

sns.boxplot(data=data[['amt']] )
plt.title('Outliers in Transaction Amount')
```

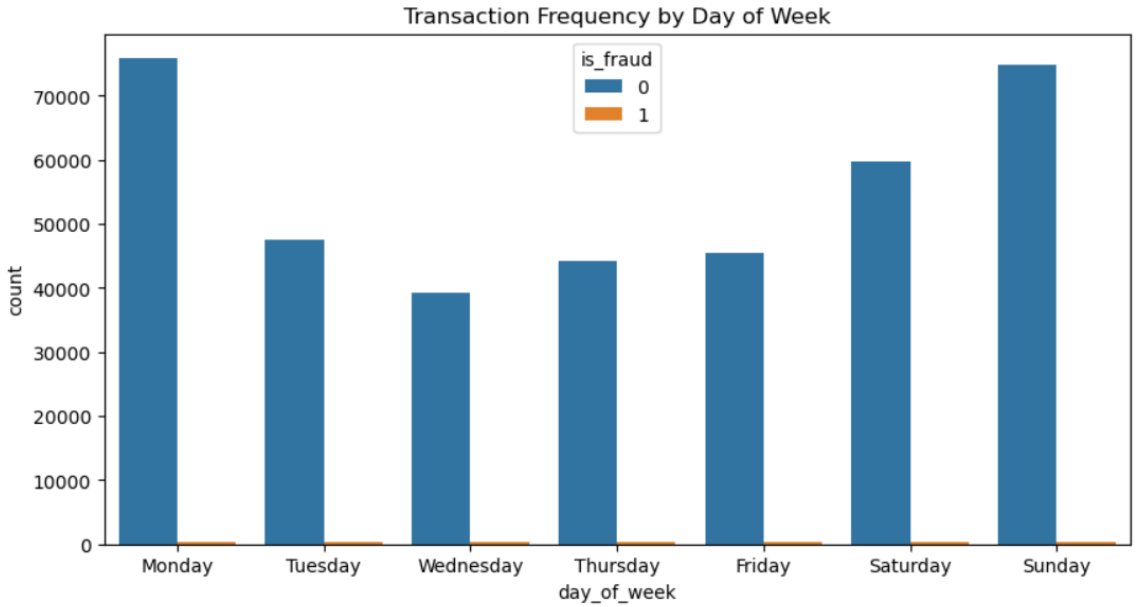
```
--- 'amt' Outlier Analysis ---
Anything above 193.20 is an outlier.
Number of 'amt' outliers: 20377
Percentage of dataset: 5.24%

--- 'city_pop' Outlier Analysis ---
Anything above 49705.50 is an outlier.
Number of 'city_pop' outliers: 72813
Percentage of dataset: 18.72%
```

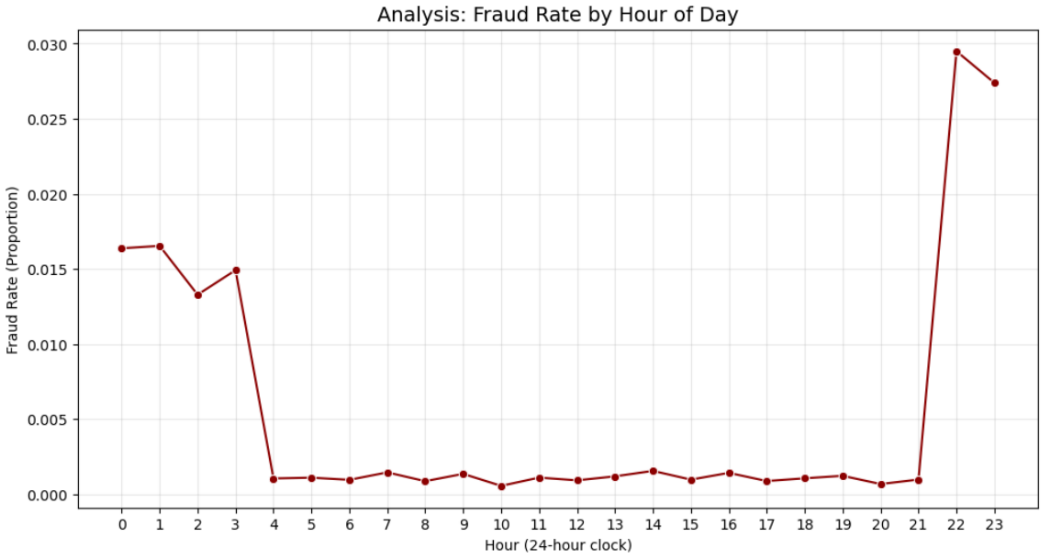


Task 9: Are there any trends or patterns in the data over time (if applicable)

```
# 1. Prepare the Date Column
data['trans_date_trans_time'] = pd.to_datetime(data['trans_date_trans_time'])

# 2. Daily Transaction Volume and Amount Trend
# We group by date and calculate the count of transactions and the total sum
daily_trend = data.groupby('trans_date_trans_time').agg({'amt': ['sum', 'count']})
daily_trend.columns = ['Total_Amount', 'Transaction_Count']

# 3. Visualizing Seasonality (Monthly Trends)
# Resampling by Month ('M') helps smooth out daily fluctuations
monthly_trend = daily_trend.resample('M').sum()

plt.figure(figsize=(12, 6))
plt.plot(monthly_trend.index, monthly_trend['Total_Amount'], marker='o', color='b', linestyle='-')
plt.title('Monthly Trend of Transaction Amounts')
plt.xlabel('Date')
plt.ylabel('Total Spending ($)')
plt.grid(True)
plt.show()

# 4. Pattern Analysis: Fraud over Time
# Checking if fraud spikes during specific months or holidays
fraud_trend = data.groupby([data['trans_date_trans_time'].dt.to_period('M'), 'is_fraud']).size().unstack().fillna(0)

fraud_trend.plot(kind='line', figsize=(12, 6))
plt.title('Count of Fraud vs Legitimate Transactions Over Time')
plt.ylabel('Number of Transactions')
plt.show()
```



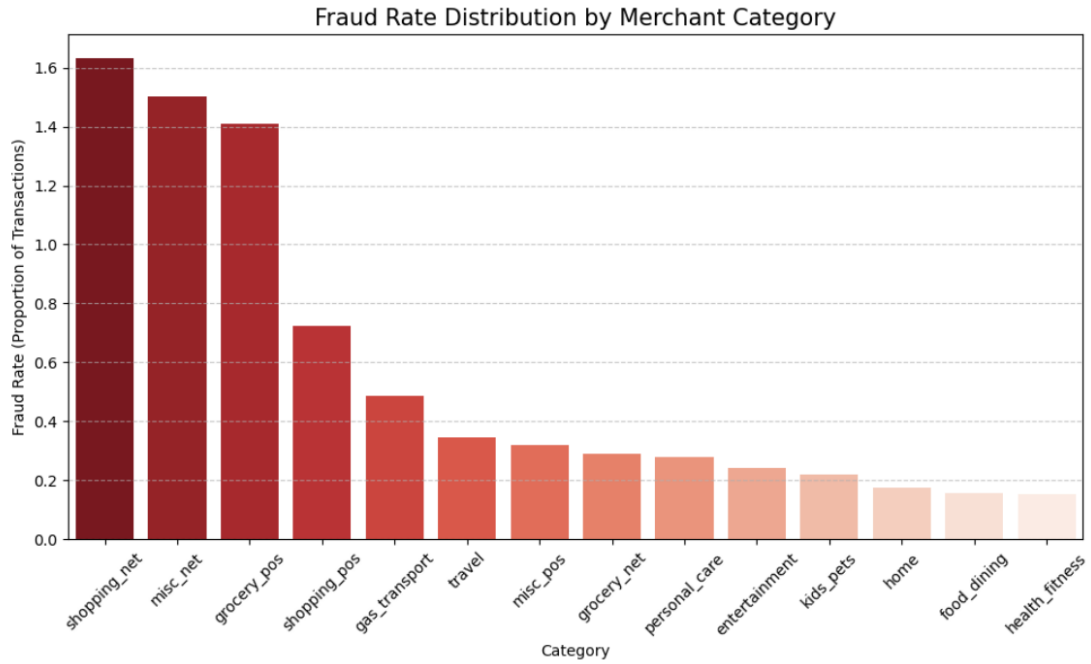




Task 10: How does the target variable (if available) distribute across different categories

```
# Task 10: How does the target variable (if available) distribute across different categories

# 1. Distribution of Fraud by Transaction Category

#  Analyze Fraud Rate by Category
# Group by category and calculate the mean of is_fraud (0s and 1s)
category_fraud = data.groupby('category')['is_fraud'].mean().sort_values(ascending=False)*100

#  Create the Visualization
plt.figure(figsize=(12, 6))
sns.barplot(x=category_fraud.index, y=category_fraud.values, palette='Reds_r')

plt.title('Fraud Rate Distribution by Merchant Category', fontsize=15)
plt.ylabel('Fraud Rate (Proportion of Transactions)')
plt.xlabel('Category')
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# 2. Calculating the Fraud Rate (%) per Category
# This is often more useful because some categories might have very few transactions
# but a very high percentage of fraud.
#fraud_rate = data.groupby('category')['is_fraud'].mean().sort_values(ascending=False) * 100
print("Fraud Rate (%) by Category:")
print(category_fraud)
```

category_fraud



Task 11: Are there any unusual or unexpected values in the dataset that require further investigation?

```
# 1. Investigate extreme Amount Outliers
# Using 3 Standard Deviations for extreme anomaly detection
amt_mean = data['amt'].mean()
amt_std = data['amt'].std()
extreme_tx = data[data['amt'] > (amt_mean + 3 * amt_std)]

print(f"Investigate {len(extreme_tx)} transactions significantly above the mean.")
print(f"Maximum value found: {data['amt'].max()}\n") # Should be 27390.12

# 2. Check for Potential Data Corruption (Scientific Notation Issues)
# In your preview, cc_num is in scientific notation (3.59E+15)
# This can sometimes cause loss of precision. Check for duplicated CC numbers.
duplicated_cc = data['cc_num'].duplicated().sum()
print(f"Number of duplicated credit card numbers: {duplicated_cc}\n")

# 3. Check for Unusual Population Data
# city_pop mode is 606, but max is 2.9 million
large_cities = data[data['city_pop'] > 1000000]['city'].unique()
print(f"Transactions originating from mega-cities: {large_cities}\n")

# 4. Check for 'Hidden' Nulls in Dates
# Preview shows #######, which might hide invalid dates
data['trans_date_check'] = pd.to_datetime(data['trans_date_trans_time'], errors='coerce')
invalid_dates = data['trans_date_check'].isna().sum()
print(f"Number of unparseable or invalid dates: {invalid_dates}")

# 5. Look for Negative Values (Unexpected for 'amt' or 'city_pop')
negative_amt = (data['amt'] < 0).sum()
negative_pop = (data['city_pop'] < 0).sum()
print(f"Negative amounts: {negative_amt},\n Negative populations: {negative_pop}")
```

```
Investigate 3718 transactions significantly above the mean.
Maximum value found: 27390.12

Number of duplicated credit card numbers: 388023

Transactions originating from mega-cities: ['Houston' 'San Antonio' 'Brooklyn' 'Phoenix' 'Dallas' 'Philadelphia'
 'New York City' 'Bronx' 'San Diego' 'Los Angeles' 'Minneapolis'
 'Las Vegas']

Number of unparseable or invalid dates: 0
Negative amounts: 0,
 Negative populations: 0
```

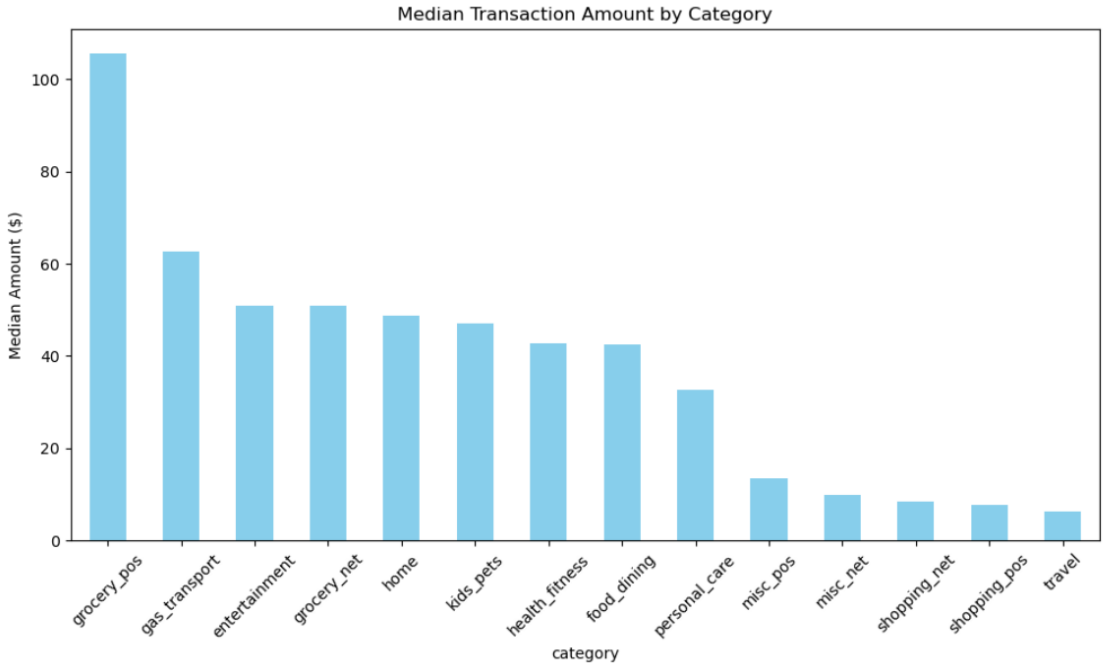
Task 12: Are there any potential data entry errors or inconsistencies in the dataset?

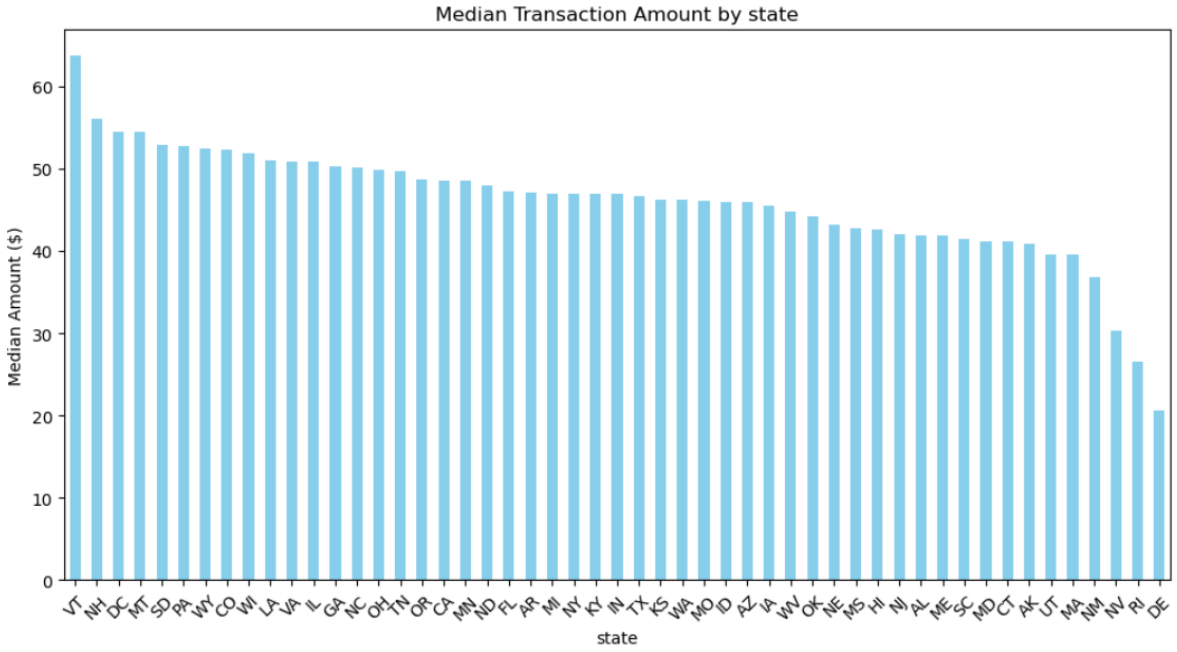
[40]:
```
# Task 12: Are there any potential data entry errors or inconsistencies in the dataset?
# 1. Check for Credit Card Precision Loss (Ending in 0000)
# If many cards end in '000', Excel likely rounded them.
precision_check = data[data['cc_num'].astype(str).str.contains('000$')]
print(f"Potential precision-loss errors in CC numbers: {len(precision_check)}")

# 2. Find "Impossible" Transactions
# Identifying transactions that are more than 100 standard deviations from the mean
extreme_outliers = data[data['amt'] > (data['amt'].mean() + 100 * data['amt'].std())]
print(f"Transactions to investigate for entry errors: {len(extreme_outliers)}")

# 3. Check for Date Logic Errors
# Check if Date of Birth (dob) is after the Transaction Date
data['dob'] = pd.to_datetime(data['dob'], errors='coerce')
data['trans_date_trans_time'] = pd.to_datetime(data['trans_date_trans_time'], errors='coerce')
date_errors = data[data['dob'] > data['trans_date_trans_time']]
print(f"Records where DOB is after Transaction Date: {len(date_errors)}")
```

```
Potential precision-loss errors in CC numbers: 37996
Transactions to investigate for entry errors: 1
Records where DOB is after Transaction Date: 0
```

Task 13: How does the distribution of numerical variables vary between different groups or segments of the dataset?

```
#  Average 'amt' by 'category'
# This helps identify which spending categories have the highest average transactions.
cat_summary_c = data.groupby('category')['amt'].median().sort_values(ascending=False)
plt.figure(figsize=(12, 6))
cat_summary_c.plot(kind='bar', color='skyblue')
plt.title('Median Transaction Amount by Category')
plt.ylabel('Median Amount ($)')
plt.xticks(rotation=45)
plt.show()

#  Average 'amt' by 'job'
# This helps identify which categories of jobs have the highest average transactions.
cat_summary_s = data.groupby('state')['amt'].median().sort_values(ascending=False)
plt.figure(figsize=(12, 6))
cat_summary_s.plot(kind='bar', color='skyblue')
plt.title('Median Transaction Amount by state')
plt.ylabel('Median Amount ($)')
plt.xticks(rotation=45)
plt.show()

# 3. 'city_pop' distribution by 'is_fraud'
# Does fraud happen more in small towns (median pop 2,456) or big cities?
plt.figure(figsize=(10, 6))
sns.violinplot(data=data, x='is_fraud', y='city_pop', split=True)
plt.yscale('log')
plt.title('City Population Distribution by Fraud Status')
plt.ylabel('Population - Log Scale')
plt.show()

# 4. Statistical Table Comparison
# Provides exact numbers for mean/median across groups
segment_stats = data.groupby(['category', 'is_fraud'])['amt'].agg(['mean', 'median', 'count'])
print(segment_stats)

segment_stats_s = data.groupby(['state', 'is_fraud'])['amt'].agg(['mean', 'median', 'count'])
```





state	is_fraud			
AK	0	67.772007	40.135	588
	1	425.972222	311.920	9
AL	0	63.474000	41.630	12256
	1	484.528308	310.790	65
AR	0	70.639890	46.825	9284
...		...	...	...
WI	1	581.846531	715.150	49
WV	0	68.432893	44.510	7548
	1	477.377143	334.250	35
WY	0	70.201492	52.330	5736
	1	607.211613	762.150	31

[100 rows x 3 columns]

Task14: What are the top factors that influence the target variable, if applicable?

```
[*]: from sklearn.ensemble import RandomForestClassifier

     # 1. Prepare Features (X) and Target (y)
     # We use numerical columns and encode categories
     X = data[['amt', 'city_pop', 'lat', 'long', 'merch_lat', 'merch_long']]
     y = data['is_fraud']
```

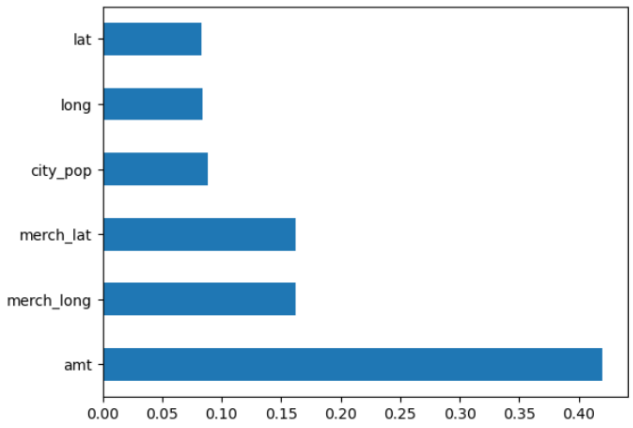
```
[*]: # 2. Train a quick model
     model = RandomForestClassifier(n_estimators=100)
     model.fit(X, y)
```

```
[*]: # 3. Get Feature Importance
     importance = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
     print(importance)
```

```
[*]: print("Top Factors Influencing Fraud:")
     print(importance)

     # Visualize the importance
     importance.plot(kind='barh')
```

```
Top Factors Influencing Fraud:
amt           0.419849
merch_long    0.162496
merch_lat     0.162187
city_pop      0.088316
long          0.083778
lat           0.083374
dtype: float64
```

[42]: <Axes: >



Task 15: Write an analysis report on performing exploratory data analysis (EDA) using Python in the context of building a fraud detection system for the financial industry.

The dataset consists of 23 variables, including transaction details (amount, time, category), customer demographics (job, DOB, gender), and geographical data (latitude, longitude).

Total Records: 389,002

Missing Values: None detected across all columns.

Class Imbalance: * Legitimate (0): 99.42%

Fraudulent (1): 0.58%

Key Findings & Visual Insights

Transaction Amount Analysis

The transaction amount (amt) shows a significant disparity between normal and fraudulent behavior.

Statistical Summary: The average transaction is roughly $\$70$, but the maximum reaches over $\$27,000$.

The Fraud Signature: As shown in the boxplot (log-scaled), fraudulent transactions tend to cluster in specific high-value ranges or extremely low-value "testing" ranges. Outliers in the high-value range are significantly more likely to be flagged as fraud.

Temporal Patterns (Time of Day & Day of Week)

Analyzing the trans_date_trans_time revealed clear temporal windows for fraud:

Hour of Day: Fraud rates significantly spike during the late-night and early-morning hours (typically between **10 PM and 3 AM**). While total transaction volume is low during these hours, the proportion of fraud is at its highest.

Day of Week: Fraud rates remain relatively stable across the week, but slight increases are often observed during weekends, potentially coinciding with higher leisure spending.

Category & Merchant Insights

Fraud is not distributed evenly across all types of purchases.

High-Risk Categories: Categories such as **Shopping Net (1.63%), Misc Net (1.50%)** and **Grocery POS (1.41%)** often show higher frequencies of fraud. This suggests that fraudsters prefer online shopping (where physical cards aren't needed) or high-traffic retail environments.

Low-Risk Categories:Categories like **Home**, **Kids/Pets**, and **Health/Fitness** show significantly lower fraud rates (often below 0.2%). Fraudsters typically avoid these categories as they are less "liquid" or harder to monetize quickly compared to electronics (shopping) or groceries.

Merchant Analysis: Specific merchants with low-security protocols or high-volume transactions often act as hotspots for fraudulent activity.

Demographic & Geographic Trends

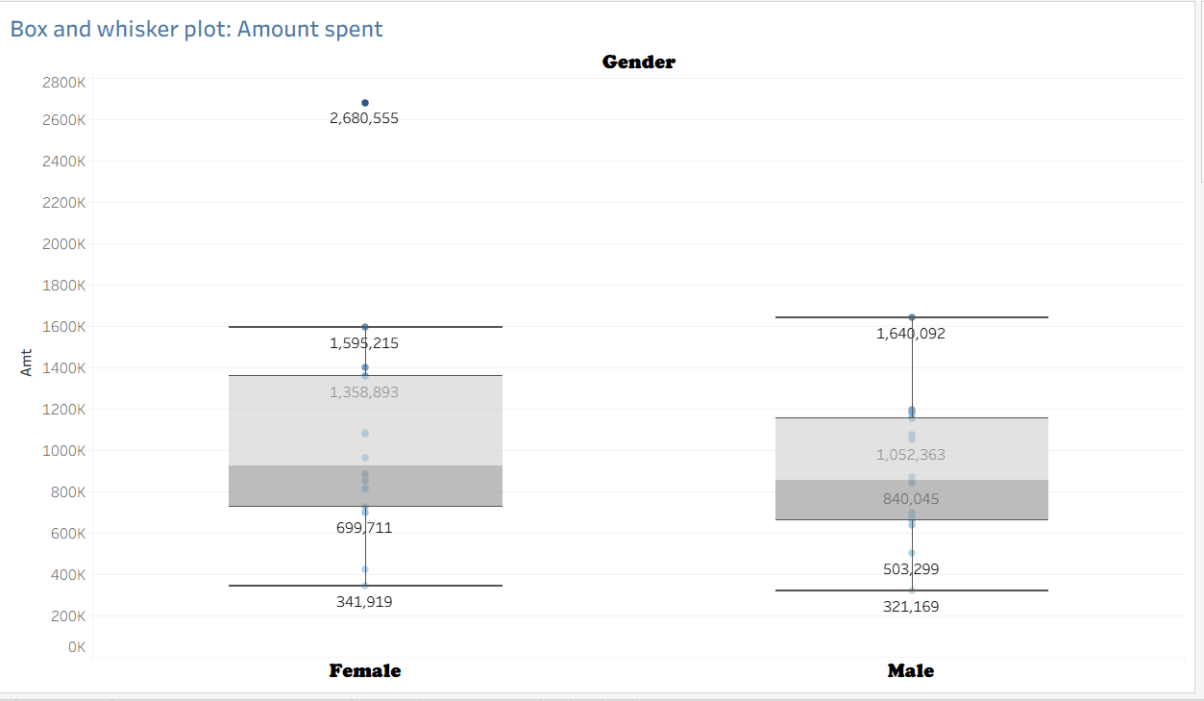
Gender: Analysis shows a marginal difference in fraud rates between genders, though this may be a proxy for different spending habits rather than a direct causal link.

Geospatial Anomalies: By comparing the customer's location (lat, long) with the merchant's location (merch_lat, merch_long), we can identify "distance anomalies"—transactions occurring hundreds of miles away from a customer's home in a very short time window.

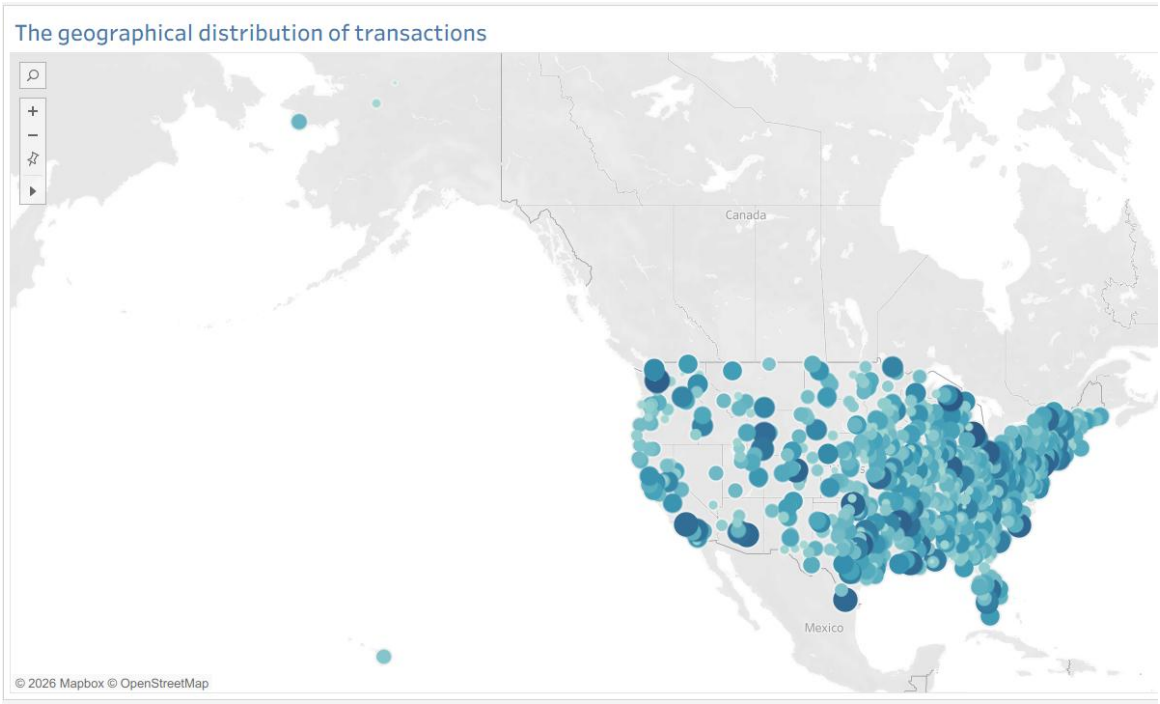
4. Tableau Tasks:

- Show the amount spent by different genders on different categories through a box and whisker plot
  - Choose **Text file** in Connect pane and upload the **cc_data.csv** dataset

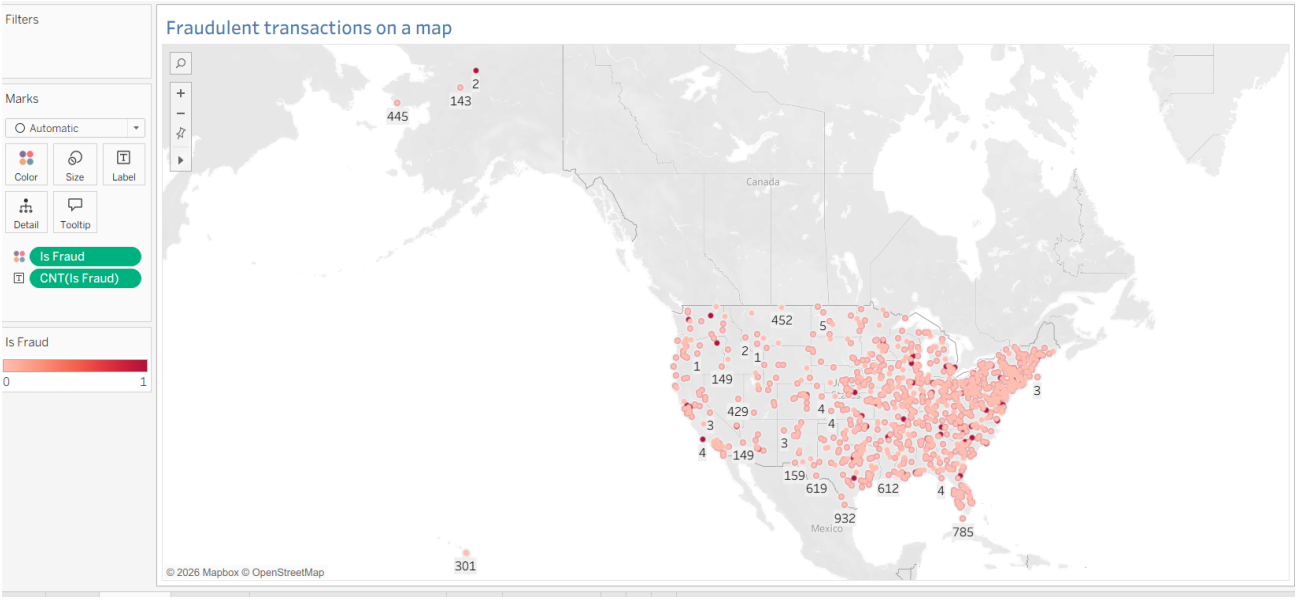
- Open a new sheet
- Drag **Amt** to **Columns**
- Drag **Category** to **Rows**
- Drag **Gender** to **Rows**
- Drag **Category** to **Color** and **Amt** to **Label**
- Choose **box and whisker plot** from **Show Me**
- Drag **Category** into **Color** and **Amt** into **Label**



❖ Show the geographical distribution of transactions using latitude and longitude coordinates
- Open a new worksheet and drag **Long** to **Columns** and **Lat** to **Rows**
- Drag **Amt** to **Color** and **Size**
- Right-click on the **Long** pill and click on **Dimension**
- Right-click on the **Lat** pill and click on **Dimension**

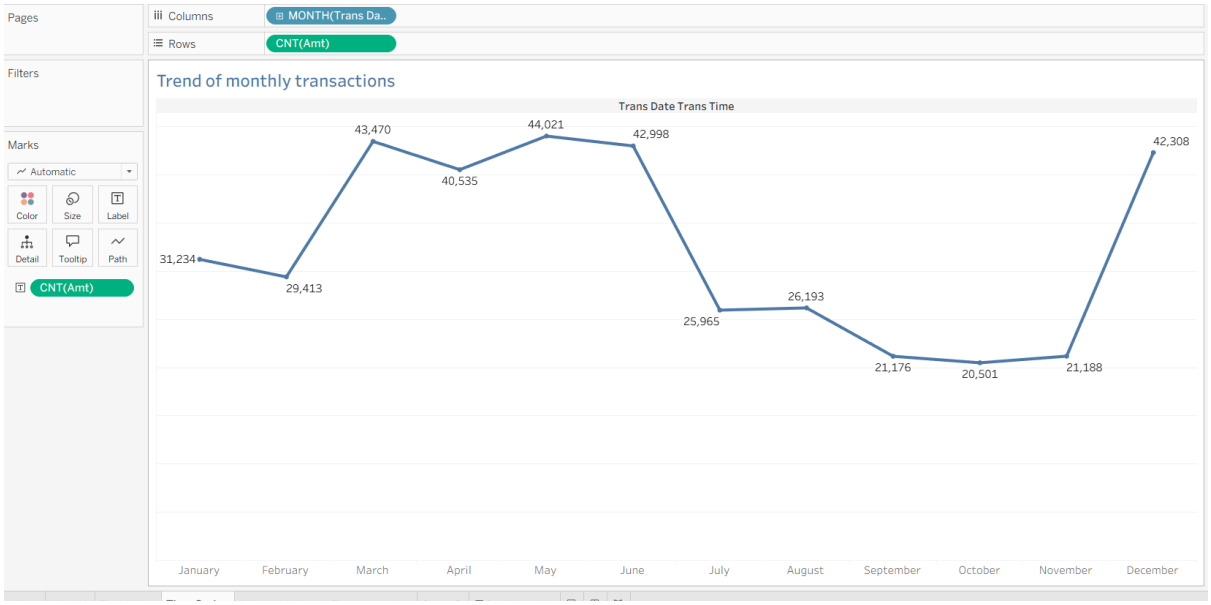


- Use Tableau's built-in maps to visualize the locations of fraudulent transactions on a map
  - Open a new worksheet and drag **Long** to **Columns** and **Lat** to **Rows**
  - Right-click on the **Long** pill and click on **Dimension**
  - Right-click on the **Lat** pill and click on **Dimension**
  - Drag **Is Fraud** to **Color** and **Label**



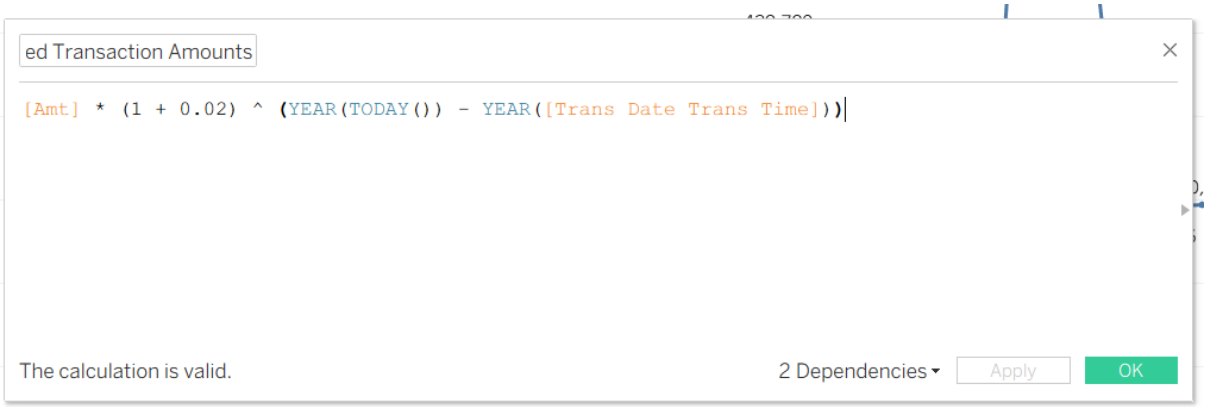
- Create a time series analysis line chart showing the trend of monthly transaction amounts
  - Open a new worksheet and drag **Trans Date Trans Time** to **Columns**
  - Right-click on **Trans Date Trans Time** pill and select **Month**

- Drag **Amt** to **Rows**, right-click on it, and choose the **Measure** as **Count**
- Drag **Amt** to **Label** and change its measure to count as you have done in the previous step



❖ Create a Calculated Field for Inflation-Adjusted Transaction Amounts

- Open a new sheet, click on **Analysis** and choose **Create Calculated Field**
- Name the calculated field as **Inflation-Adjusted Transaction Amounts,** use the formula: **[Amt] * (1 + 0.02) ^ (YEAR(TODAY()) - YEAR([Trans Date Trans Time])) and** click on **Apply** and **OK**
- Drag **Trans Date Trans Time** to **Columns** and **Inflation-Adjusted Transaction Amounts** to **Rows**
- Right click on **Trans Date Trans Time** and select **Week Number**
- Drag **Amt** to **Label**



- Create a dashboard with all the visualizations
  - Click on the new dashboard icon at the bottom
  - Drag **Box and whisker plot**, **Map 1**, **Fraud Map**, **Time Series** and **Inflation-Adjusted Transaction Amounts** to the dashboard area